Tanmark Industries: from paper to one system

How a ten-department manufacturer moved order to cash, purchasing and accounting onto one system, and the decisions that made it work

A product owner case study. Tanmark Industries is a fictional company; the design patterns are real ones from order-processing and accounting systems.

Every timing and rate in this report is illustrative, from a synthetic dataset. One outcome is real, and it is labelled.

September 2026

Summary

Tanmark Industries is fictional. It was built to show the shape of a real change without using any real client's data. Every number below is illustrative, from the synthetic dataset, unless it is labelled real.

Tanmark builds truck-mounted tanks, steel drums and containers to order, and services vehicles. Its 10 departments ran order to cash, purchasing and accounting on paper forms, email and spreadsheets. Its customers are businesses that buy on credit. Nobody could see where an order was, and stock was checked by walking the shelves. Overdue customers surfaced only at the quarter-end statements, and accounting vouchers were edited and deleted after the fact.

The product owner led the move to one back-office system on one data model. A team of six developers built it. Every document got a lifecycle: named states, the event that moves it, the role allowed to move it, and the moves the system must refuse.

What the case study shows:

1. **A state model sits at the centre.** Seven documents, 44 allowed moves, 19 refused ones, and ten rules that always hold. Every error path in the user stories is a move the model refuses, and the build checks it.
2. **Seventeen decisions shaped it, and the business made the business's ones.** Goods leave only with their invoice, one per trip (D16). Customers buy on 30-day terms, and the chief accountant releases any order for a customer over its limit (D17). Purchase approval stays on paper (D9). Posted vouchers never change, and nothing is ever deleted (D10).
3. **The illustrative results.** The median time from order to delivery fell from 24.0 to 17.8 days. Overdue receivables fell from 34% of the balance to 18%. Monthly rework averaged 9.8% after pilot go-live, down from 17.5%.
4. **What went wrong.** Collection stayed on paper through the pilot, adoption stalled on the night shift, and three decided rules were missing from the first build. Each became a rule in the rollout.

The real outcome, and the only one. The system this case study is modelled on is used by 50 to 100 people. It replaced hand record keeping, manual warehouse checks and manual order processing. No before-and-after timings were measured there, so none are claimed.

1. The problem

A customer phoned or emailed an order, and sales wrote it on a paper form. Logistics checked stock by walking the shelves against a paper ledger. When materials were short, a buyer emailed a supplier and noted the order in a spreadsheet. The workshop built from a paper job card, and the warehouse shipped with a paper delivery note.

Accounting keyed each invoice by hand, sometimes after the truck had gone, and matched payments to invoices by hand. Customers paid on credit terms kept in a binder, so nobody saw who was overdue until the quarter-end statements went out. Vouchers could be edited, or deleted, after they reached the ledger. About 200 orders a month went through this path.

Four things followed from it:

- **No one could see where an order was.** A customer's question became a round of phone calls.
- **Stock was not trusted.** Orders were "cancelled" after materials had already gone to the workshop, and the stock record never recovered.
- **Receivables were seen late.** In the year before pilot go-live, 34% of open receivables were past due on average, and invoices followed delivery by a median of 3.8 days.
- **The ledger could not be audited.** The voucher numbers had gaps nobody could explain. In the twelve months before pilot go-live, an average of 17.5% of orders needed rework, and 9.7% carried an error.

2. The role and its boundary

The product owner owned what the system did, the order it was built in, and how it was rolled out. The product owner ran discovery with every department head, modelled every document's lifecycle, and wrote the requirements and the acceptance scenarios.

A team of six developers built it: a tech lead, four engineers and a security engineer. The product owner owned the backlog and its order; the executive sponsor, the director, reviewed the roadmap every week.

The boundary mattered as much as the role. Buyers decided what to buy, and the logistics manager signed purchase orders. Finance decided the posting and correction policy, which status combinations were legal, each contract's payment terms, and when an order on credit hold may go ahead. The sales and logistics leads decided how an order may close. The product owner's job was to make those decisions enforceable. The RACI in `raci/raci.xlsx` checks this row by row: the product owner holds no R or A on any of its ten business decisions.

3. Constraints

- **People new to shared systems.** Training had to be short, role-based and on every shift.
- **Paper as the trusted record.** Paper could only stop once the system had earned it.
- **History of mixed quality.** Open orders had to load in the state they were really in.
- **Ten departments with ten ways of working.** Each would change a process it had run for years.
- **A small team.** Six developers and one product owner, so every feature had to earn its place.

4. The process, before and after

Three flows: order to cash, procure to pay, and the accounting that closes both.

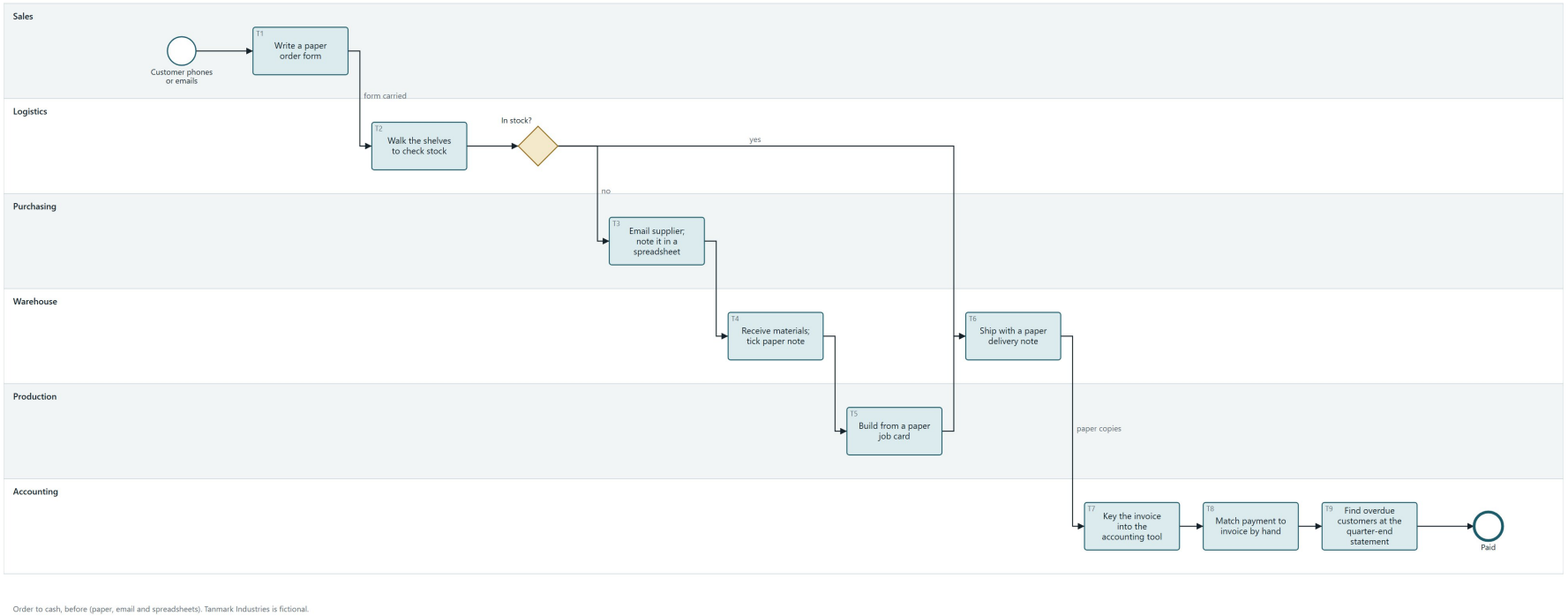


Figure 1. Before: orders on paper forms, stock checked by walking the shelves, purchases by email, invoices keyed and matched by hand. BPMN, process/before.bpmn.

The after map adds what the old process could not do. An order request becomes an order only when confirmed, and a customer on credit hold needs the chief accountant's release first. Sales editing locks at the handoff to logistics. Receiving a purchase order counts as its approval, and one receipt feeds both production and supplier payment. Each trip is issued by the warehouse, invoiced by accounting, and only then dispatched, so goods never leave without their invoice. The order completes when its last trip is delivered; collection then runs on the contract's credit terms, and every path to payment passes a posted voucher. Two closures show D6: a withdrawn order is cancelled before production, and an order no longer wanted after production closes as not completed.

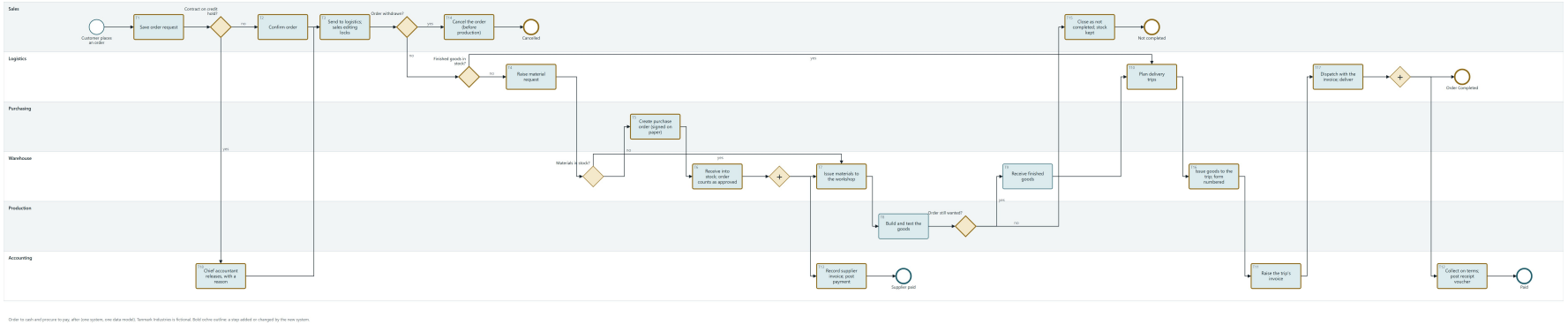


Figure 2. After: one system from order request to posted receipt, with purchasing feeding production and supplier payment in parallel. Outlined steps are new or changed. BPMN, process/after.bpmn.

Some work is not part of any one order's journey, so it has a map of its own. Each week the ageing view finds overdue invoices, and sales chases those customers. Each month the warehouse count is reconciled with the books. Each quarter every customer signs a statement of its open invoices. Each new supplier price list is loaded a week before it takes effect.

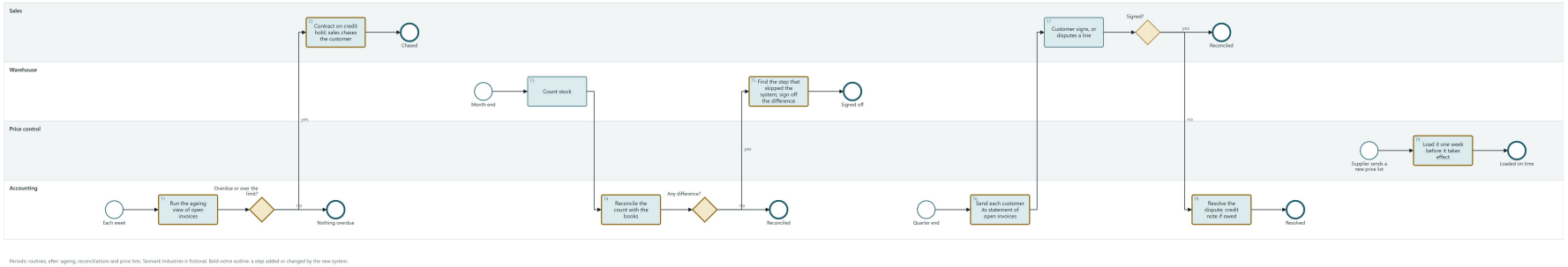


Figure 3. Periodic routines: the weekly ageing and chase, the monthly stock reconciliation, the quarterly customer statement, and loading supplier price lists. BPMN, process/periodic.bpmn.

5. The state model

Every document has a lifecycle, declared once in `state/model.py`. The diagrams, the transition table and the refused moves are generated from it, and the stories are checked against it. The sales order is the richest:

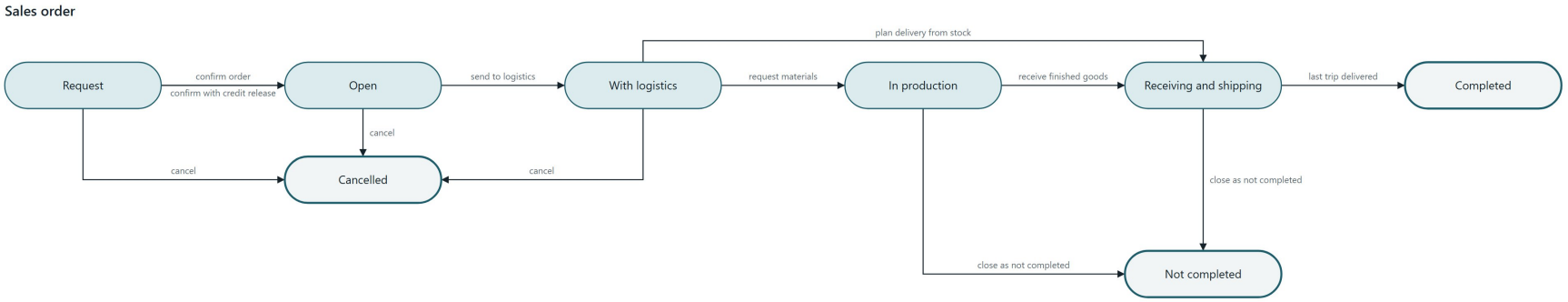


Figure 4. Sales order: a request becomes an order when confirmed; it moves forward only; it can be cancelled only before production, and after that it closes as Not completed.

The ten rules that always hold:

Rule	What it says
R1	No document is ever deleted. Every closure is a state, so each record and its number survive.
R2	A posted voucher or a raised invoice never changes. A correction is a reversing voucher or a credit note.
R3	Nothing that has moved stock is cancelled. An order in production closes as Not completed; a received purchase order is returned.
R4	A delivered trip is read-only, and a closed order freezes itself and all its trips.
R5	The warehouse issues exactly the planned quantity, or the plan is revised first.
R6	After an order goes to logistics, sales cannot edit it. A quantity may fall; a rise is a new order.
R7	Roles grant access to data; the transition table grants the right to move a document.
R8	A stock form gets its official number only when the stock moves. Before that it is a draft with no number.
R9	Goods leave only with their invoice: each trip has one invoice, raised before the trip is dispatched.
R10	Sales are on credit terms set per customer and contract, 30 days unless the contract says otherwise. A customer over its credit limit, or with an invoice past due, gets no new order unless the chief accountant releases it.

The sales invoice carries the receivable. It is raised for an issued trip, paid in one or several allocations, or credited if the trip comes back:

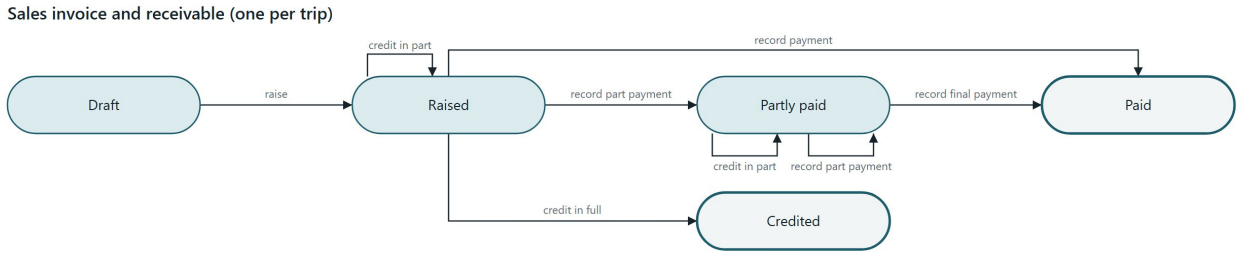


Figure 5. Sales invoice: Draft, then Raised for an issued trip; Partly paid and Paid as receipts are allocated; Credited by a credit note if the trip is returned.

The accounting voucher shows the save-versus-post pattern in four states:

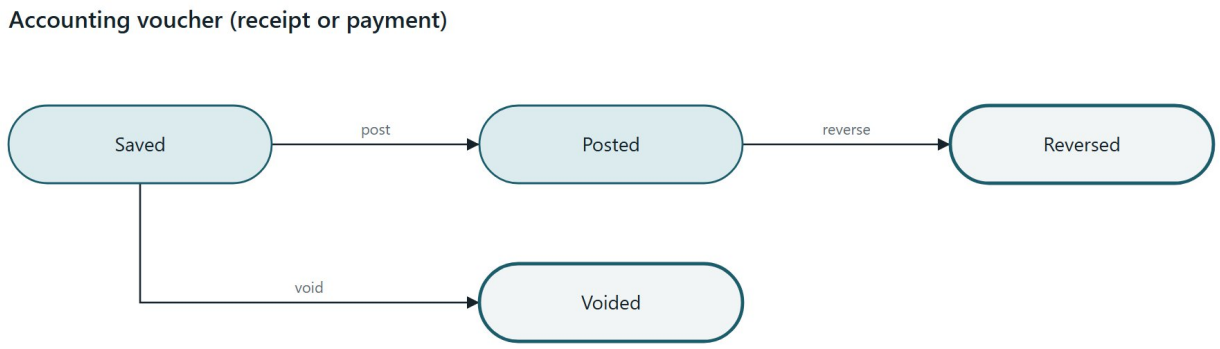


Figure 6. Accounting voucher: Saved, then Posted and locked; a posted voucher is corrected by a reversal; a saved one can be voided; none is deleted.

The model also carries a **decided-versus-built register**: each rule decided in a meeting, checked against the build in UAT. It found three gaps before the parallel run (illustrative): vouchers with no posted status, sales still able to edit after handoff, and account permission checks switched off during the build.

6. The decisions that shaped it

The full log is in `decisions/decision-log.md`: each decision with its context, the option rejected and who decided.

ID	Decision	Who decided
D1	One data model for sales, purchasing, stock and accounting	Product owner and tech lead; sponsor signed off
D2	A pilot group before full cutover	Product owner; sponsor accepted the longer timeline
D3	Migrate what is open, by its state; archive what is closed	Product owner with the department leads
D4	Roles grant data; a transition table grants moves	Product owner and security engineer
D5	An order request becomes an order only when confirmed	Product owner with the sales lead
D6	A forward-only order lifecycle with two ways to close	Sales and logistics leads
D7	The order locks for sales when it goes to logistics	Sales and logistics leads
D8	The sale carries three statuses, and they move in one order	Product owner proposed; finance ruled the combinations
D9	Purchase approval stays outside the system	Executive sponsor with the logistics manager
D10	Save is not post, and nothing is ever deleted	Finance lead and chief accountant
D11	Acceptance is written before the build, and signed by the business	Product owner; department heads agreed
D12	Paper is retired only after a clean parallel run	Product owner with each department head
D13	Every decided rule is tested against the build	Product owner
D14	Physical facts guard the documents: delivered trips read-only; ship exactly the plan	Logistics lead, with the warehouse lead
D15	The warehouse reviews a draft stock form before stock moves; the number comes with the movement	Warehouse lead; the product owner kept the number tied to the movement
D16	The invoice travels with the goods: one invoice per trip	Chief accountant with the logistics lead
D17	Sales are on credit terms, with a credit hold the chief accountant releases	Chief accountant with the director

D2, a pilot before cutover. The rejected option was going live everywhere on one date. It has a public reference case in Canada's Phoenix pay system, where a pilot was cut and warnings did not travel up. The source is the Office of the Auditor General of Canada, 2018 spring reports, Report 1.

D9, approval outside the system. An in-app approval step was designed, and the buyers asked what it would add to a signature they already collected. The trade-off is stated in the log: the system cannot show who approved an order, so the control lives in the paper file and the receiving count.

D15, review before an action that cannot be undone. The product owner designed the stock form to download after Issue or Receive. Warehouse keepers wanted to check it before the click, and accepted the extra step; that control was theirs to decide. The product owner agreed, and held one line. A form downloaded before the click is a draft with no number, because a numbered form would record a movement that had not happened.

D8, three statuses in one order. Finance asked for "three ways" to record a sale. Mapped against the documents, the three were three outcomes: invoiced, delivered, paid. The first draft treated them as independent; finance's ruling showed they are one sequence, because a trip leaves only with its invoice and customers pay on credit after delivery. Ten of twenty-seven combinations are legal, and the rest are refused or flagged.

D17, credit terms with a hold someone owns. Most of what finance manages here is receivables. Terms and a credit limit live on each contract; an invoice's due date comes from them; an ageing view shows what is overdue each week instead of each quarter. A customer over its limit or overdue gets no new order unless the chief accountant releases

that order, with a reason. A warning sales could click past was rejected: the hold works only because releasing it is a decision someone owns.

7. Requirements and acceptance

Every story has Gherkin scenarios agreed before the build. **Every error path is a move the state model refuses**, and `uat/build_uat.py` refuses to build the test script if one is not.

Story	Scenarios	Its error path: the refused move	Signed off by
Order lifecycle	6	Request: confirm on credit hold; With logistics: sales edit	Sales
Order closure	4	In production: cancel	Sales, with logistics
Delivery trip	6	Planned: issue a different quantity; Planned: download numbered form; Issued: dispatch without an invoice	Logistics
Material request	6	Issuing: cancel; Open: download numbered form	Factory manager
Purchase order	7	Received: cancel; Open: download numbered form	Logistics manager
Voucher posting	5	Posted: edit; Saved: delete	Chief accountant
Receivables	7	Draft: raise before the trip is issued; Raised: edit amount	Chief accountant
Sale statuses	6	Not invoiced: deliver before invoicing	Finance lead

The UAT script in `uat/uat-script.xlsx` is generated from the stories: 54 rows, each carrying its document, its refused move, its map steps and its decisions. The traceability matrix (W8) goes one step up, tying every test to a business objective.

8. The working papers

Beside the nine deliverables are thirteen working papers, the documents a business analyst and a project manager keep while the work is done.

Paper	What it answers
W1 Charter and scope, with a context diagram	What is in scope, and why?
W2 Stakeholder register and map	Who had to be won over?
W3 Elicitation log	How were requirements gathered and confirmed?
W4 Data model and data dictionary	What does the business record, and which fields carry the rules?
W5 Non-functional requirements	What must hold across the whole system, and how is it tested?
W6 Business rules and a decision table	Which rules apply when, and which status combinations are legal?
W7 Interface: sales to accounting	Where one application's number becomes another's document
W8 Requirements traceability matrix	Which test proves which objective? (generated)
W9 Backlog and change requests	What came first, and how was a change after sign-off handled?
W10 RAID log	Risks, assumptions, issues and dependencies
W11 Business case	Why this option, and what it returns (generated)
W12 Communications and lessons learned	Who heard what, when, and what changes next time
W13 Release roadmap and dependencies	What ships when, what each release waits on, and the dependency that slipped

Two show the work that decides a case like this. **W3**, the elicitation log, records how the questions "when may this still change, and who may change it?" produced most of the rules. **W6** holds finance's ruling on which combinations of the three sale statuses are legal: ten of twenty-seven, in one sequence.

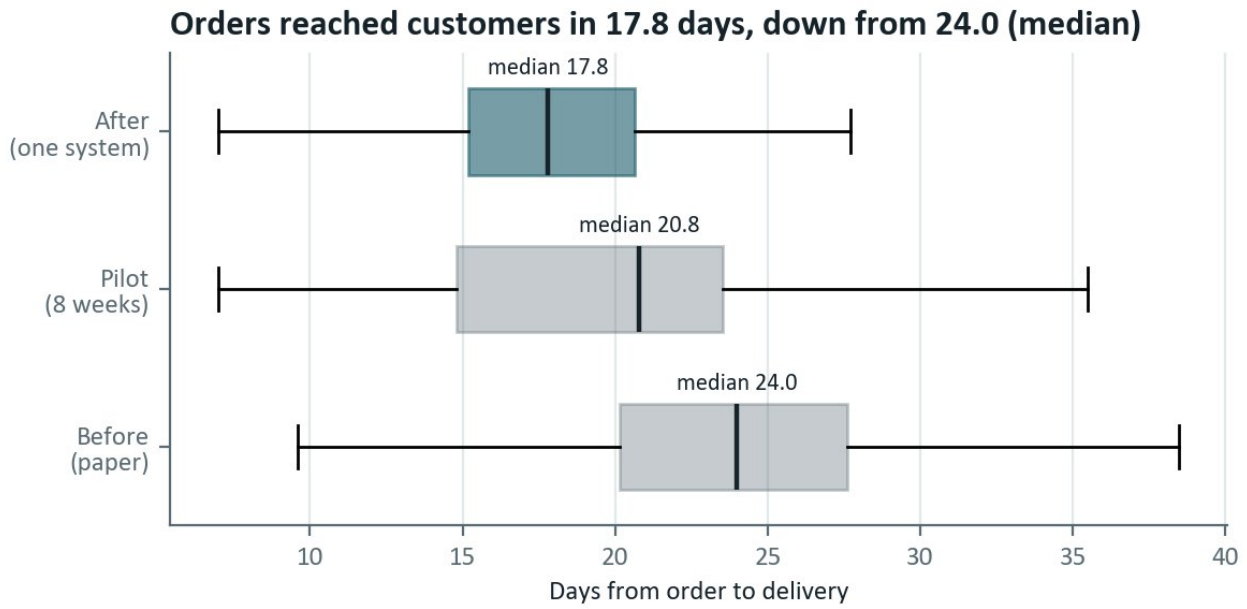
9. Rollout

The one-page plan is `docs/cutover-plan.pdf`. In short:

- **Three phases.** A pilot of 24 people in sales, logistics and purchasing, the warehouse and the drum workshop for 8 weeks, while open records migrated. Then a one-month parallel run for every department, then the system as the record.
- **Migration by state.** Each open order loaded in the state it was really in, agreed with its department; each customer's opening receivable had to equal its last signed statement. Posted history loaded as posted (D3).
- **Customers told first.** The credit hold is the change customers feel. Each heard about it from their sales contact four weeks before the parallel run, and each customer on the hold list got a call the week before (W12).
- **Training.** A superuser in each department, one-hour sessions per role on every shift, and a card per role.
- **Parallel run.** One month, every department, until two weeks passed with no open discrepancy (D12). Paper stays only for the signed purchase order and the customer-signed quarterly statement.
- **Go / no-go.** Six criteria before each phase, including the transition table and no open decided-versus-built gap.

10. Results

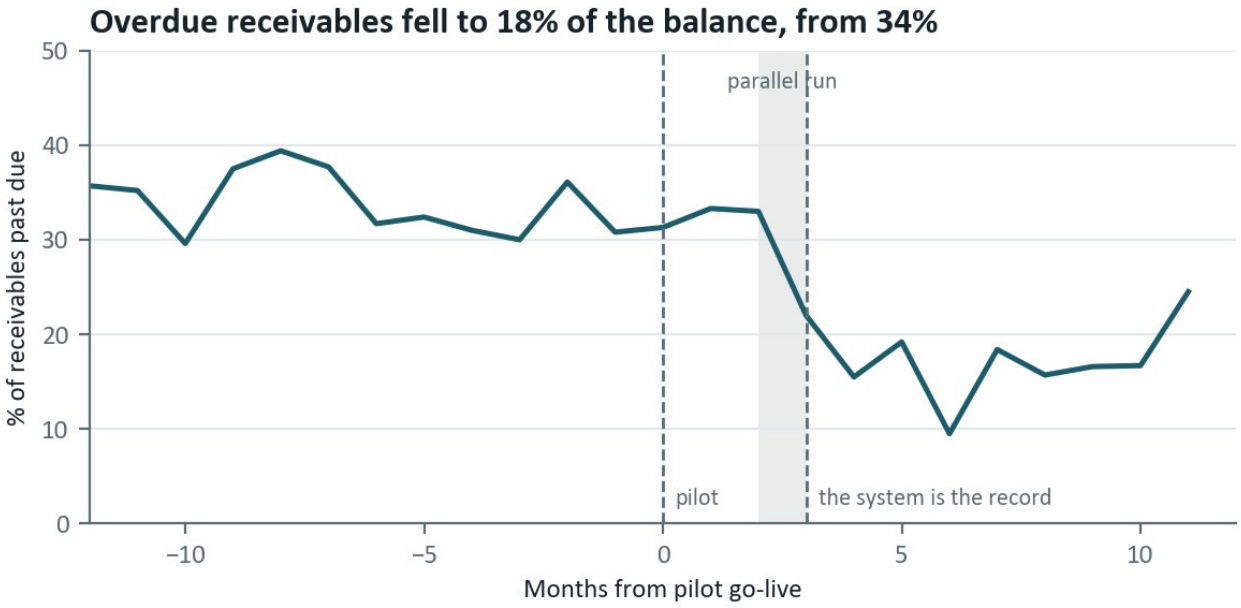
Every number in this section is illustrative, from the synthetic dataset. The dataset is in `data/`, and `data/generate_dataset.py` regenerates it. Weeks and months count from **pilot go-live**; the system became the record at **full go-live**, in week 13.



Source: synthetic dataset, data/generate_dataset.py (seed 20260929). Illustrative, not measured. Denominator: 300, 80 and 300 orders.

Figure 7. Lead time: the median order reached its customer in 17.8 days after the pilot, against 24.0 before. The pilot sat between the two at 20.8.

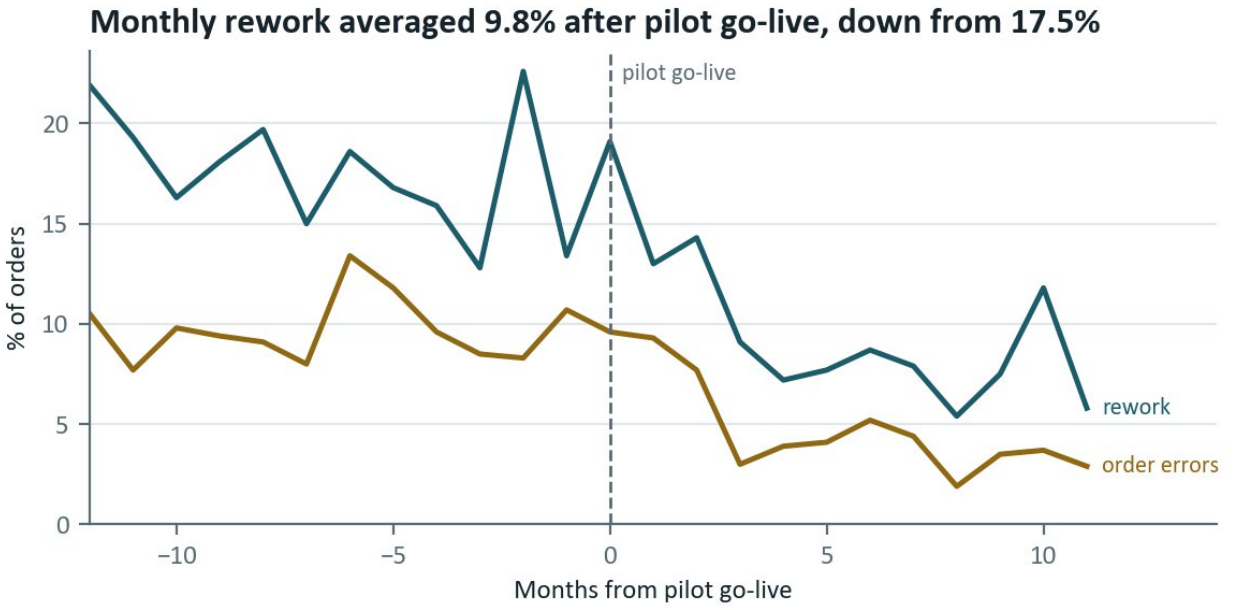
The median time from order to delivery fell by 26%, from 24.0 days to 17.8. The medians are over 300 orders before pilot go-live and 300 after the pilot, and 80 in the pilot. Invoices, which had followed delivery by a median of 3.8 days, now leave with the goods.



Source: synthetic dataset, data/generate_dataset.py (seed 20260929). Illustrative, not measured. Denominator: open receivables at each month end.

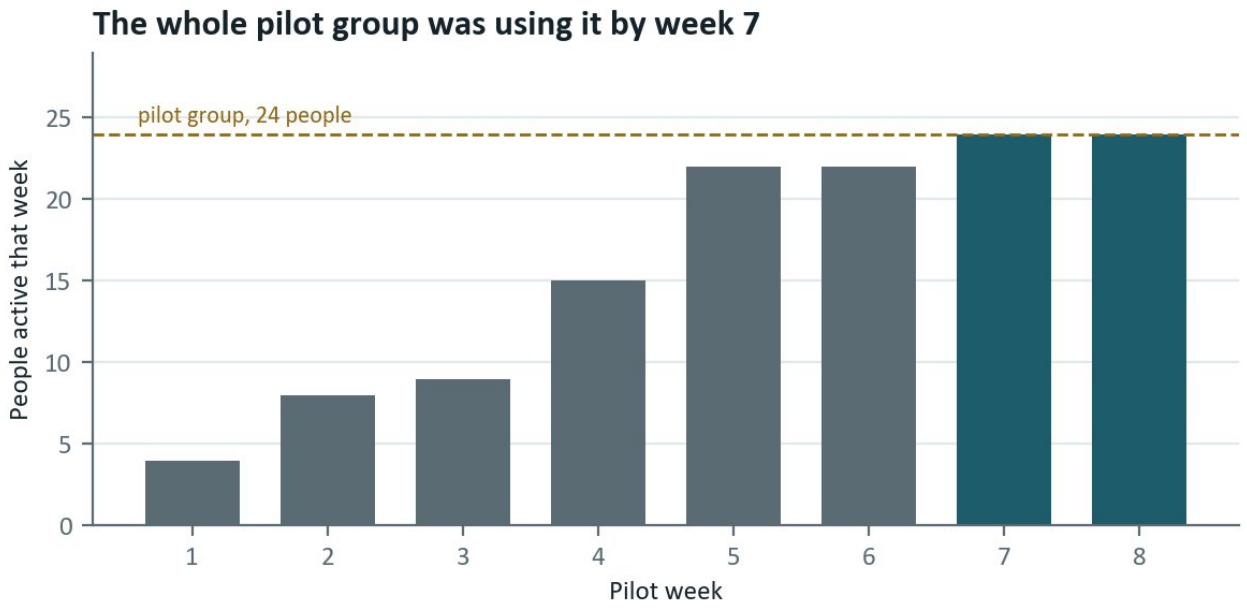
Figure 8. Overdue receivables by month: about 34% of the open balance was past due before pilot go-live and through the pilot, and 18% on average once receivables moved into the system in the parallel-run month.

Overdue receivables fell from 34% of the open balance to 18%, and days sales outstanding from 57 to 41, against 30-day terms. They did not move in the pilot, which is the next section's first finding.



Source: synthetic dataset, data/generate_dataset.py (seed 20260929). Illustrative, not measured. Denominator: orders raised each month.

Figure 9. Rework and order errors by month. Monthly rework averaged 9.8% after pilot go-live, down from 17.5%; order errors averaged 4.9%, down from 9.7%.



Source: synthetic dataset, data/generate_dataset.py (seed 20260929). Illustrative, not measured. Denominator: the 24-person pilot group.

Figure 10. Pilot adoption: people in the 24-person pilot group active each week. Everyone was using the system by week 7.

What is real. The real system behind this case study is used by 50 to 100 people, and it replaced hand record keeping, manual warehouse checks and manual order processing. That is the only real outcome claimed here.

11. What went wrong

Collection stayed on paper through the pilot. Pilot trips left with invoices raised in the system, but receipts were still matched by hand and terms still lived in the binder. Overdue receivables in the pilot months averaged 32%, against 34% before: no change. They fell only when collection moved into the system in the parallel run.

Adoption stalled. The pilot group reached 22 of 24 active users in week 5 and stayed there until week 6. The two missing people worked the night shift in receiving and had been trained in a day session. A session on their shift fixed it by week 7.

Three decided rules were missing from the first build. Vouchers had no posted status, so saved and posted looked the same. Sales could still edit an order that logistics had. Account permission checks had been switched off during the build. UAT found all three, because every decided rule had its own test (D13).

Rework peaked in the pilot's first month. At 19.1%, that month had more rework than the old average of 17.5%, while the pilot group learned the system on live orders. From the second month on it stayed below the old average.

Rework rose again in month 10. A supplier's new price list was loaded two weeks late, so orders placed in that window were re-keyed at the new prices. Monthly rework rose to 11.8% before falling back.

12. What the product owner would do differently

- **Put collection in the pilot.** A pilot that stops at the invoice tests most of the process and none of its end. In a business that sells on credit, the end is where the money is.
- **Train by shift as well as by role.** The stall came from who was on shift that week.
- **Start discovery with the state model.** Asking when each document may still change, and who may change it, produced most of the rules.

13. Limits of this case study

- **Tanmark Industries is fictional.** Its departments, people and numbers were built to be realistic, and none of them was measured.
- **The design patterns are real** ones from order-processing and accounting systems. No client, product, code or data appears here.
- **The results are a synthetic dataset.** It shows how the change would be measured. What it demonstrates is the method.
- **One outcome is real**, and it is stated as it is on record: 50 to 100 users, replacing hand records, manual warehouse checks and manual order processing. No real timings exist, so none are claimed.

Files in this repository

Path	What it is
`state/state-model.md`	The state model: diagrams, transitions, refused moves, rules, register
`decisions/decision-log.md`	The seventeen decisions, each with the option rejected and who decided
`process/before.bpmn`, `process/after.bpmn`, `process/periodic.bpmn`	The process maps, BPMN 2.0 with layout
`stories/*.feature`	Eight user stories, 54 Gherkin scenarios
`uat/uat-script.xlsx`	The UAT script and business sign-off
`docs/cutover-plan.pdf`	The one-page readiness and cutover plan
`raci/raci.xlsx`	The RACI across ten departments
`deck/interview-deck.pptx`	The ten-minute interview deck
`working-papers/`	Thirteen BA and PM working papers
`data/`	The synthetic dataset and its generator